Optimal Trajectory Tracking for Differential-Drive Robots via CEC and Generalized Policy Iteration

1st Armon Barakchi
abarakchi@ucsd.edu
A16950466

I. Introduction

In the world of autonomous mobile robots, a key task is tracking desired trajectories while avoiding obstacles. This problem may be challenging due to nonlinear dynamics, motion noise, and the need to satisfy safety constraints in real-time. Some examples of this problem in the real world are robots operating in warehouses, autonomous vehicles, or search and rescue missions. In this project, we develop and compare two methods of performing reference tracking for a differential-drive robot tasked with tracking a figure-eight (lemniscate) reference trajectory while avoiding 4 circular obstacles, two of which are placed directly in the reference trajectory path. The robot's motion is subject to gaussian noise and the control input is constrained to a bounded set of linear and angular velocities. We formulate the objective as a discounted infinite horizon stochastic optimal control problem over a tracking error state space and evaluate two approaches: Certainty Equivalent Control (CEC) and Generalized Policy Iteration GPI. CEC approximates the stochastic problem as deterministic and solves a finite horizon nonlinear program online, while GPI discretizes the error state space into a grid and computes an optimal policy offline. We present three experiments for each that characterize how key parameters of each algorithm effect tracking accuracy, computational cost, and collision avoidance. The paper ends with a head-to-head comparison of each algorithm's best parameter configurations.

II. Problem Formulation

We consider a differential-drive robot with state $\mathbf{x_t} := (\mathbf{p_t}, \theta_t)$, where $t \in \mathbb{N}$ is a discrete time index, $\mathbf{p_t} \in \mathbb{R}^2$ is the position, and $\theta_t \in [-\pi, \pi)$ is the orientation. The robot is controlled by a linear and angular velocity, $\mathbf{u_t} := (v_t, w_t)$, where $v_t, w_t \in \mathbb{R}$.

The motion of the robot is governed by a discrete-time kinematic model obtained via exact integration of the continuous-time motion model with time interval $\Delta > 0$:

$$\begin{aligned}\mathbf{x}_{t+1} &= \begin{bmatrix}\mathbf{p}_{t+1}\\ \theta_{t+1}\end{bmatrix} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t)\\ &= \begin{bmatrix}\mathbf{p}_t\\ \theta_t\end{bmatrix} + \begin{bmatrix}\Delta\,\mathrm{sinc}\left(\frac{\omega_t\Delta}{2}\right)\cos\left(\theta_t + \frac{\omega_t\Delta}{2}\right) & 0\\ \Delta\,\mathrm{sinc}\left(\frac{\omega_t\Delta}{2}\right)\sin\left(\theta_t + \frac{\omega_t\Delta}{2}\right) & 0\\ 0 & \Delta\end{bmatrix}\begin{bmatrix}v_t\\ \omega_t\end{bmatrix} + \mathbf{w}_t,\\ &\quad t = 0, 1, 2, \ldots\end{aligned} \tag{1}$$

where $\mathbf{w_t} \in \mathbb{R}^3$ models the motion noise with Gaussian distribution $\mathcal{N}(\mathbf{0}, diag(\boldsymbol{\sigma})^2)$ with $\boldsymbol{\sigma} := [0.04, 0.04, 0.004]^T \in \mathbb{R}^3$. We also assume that $\mathbf{w_t}$ is independent across time and of the robot state $\mathbf{x_t}$. The kinematics above define the probability density function $p_f(\mathbf{x_{t+1}}|\mathbf{x_t}, \mathbf{u_t})$ as a Gaussian distribution with mean $f(\mathbf{x_t}, \mathbf{u_t}, \mathbf{0})$ and covariance $diag(\boldsymbol{\sigma})^2$. Lastly, we constrain the control input to $\mathcal{U} := [0.1, 1] \times [-1, 1]$.

The goal is to design a control policy that drives the robot to follow a reference trajectory $(\mathbf{r}_t, \alpha_t)$ — a position in $\mathbb{R}^2$ and a heading in $[-\pi, \pi)$ — while keeping it clear of four circular obstacles $\mathcal{C}_i$ of radius 0.5 centered at $(\pm 2.35, \pm 0.95)$ and $(\pm 1.0, 0)$. See Figure 1 for a visualization of this objective. The robot's body is modeled as a Euclidean ball $\mathcal{B}_{0.3}(\mathbf{p_t}) := \{z \in \mathbb{R}^2 | \|z - \mathbf{p_t}\|_2 \leq 0.3\}$, leading to the following free-space description of the environment:

$$\mathcal{F} := [-3, 3]^2 \setminus \bigcup_{\mathbf{q} \in \bigcup_{i=1}^4 \mathcal{C}_i} \mathcal{B}_{0.3}(\mathbf{q}).$$

Rather than working directly in the robot's state space, we define a tracking error state $\mathbf{e}_t := (\tilde{\mathbf{p}}_t, \tilde{\theta}_t)$ where $\tilde{\mathbf{p}}_t = \mathbf{p}_t - \mathbf{r}_t$ and $\tilde{\theta}_t = \theta_t - \alpha_t$ capture how far the robot deviates from the reference in position and heading. The error evolves as:

$$\begin{aligned}\mathbf{e}_{t+1} &= g(t, \mathbf{e}_t, \mathbf{u}_t, \mathbf{w}_t)\\ &= \begin{bmatrix}\tilde{\mathbf{p}}_t\\ \tilde{\theta}_t\end{bmatrix} + \begin{bmatrix}\Delta\,\mathrm{sinc}\left(\frac{\omega_t\Delta}{2}\right)\cos\left(\tilde{\theta}_t + \alpha_t + \frac{\omega_t\Delta}{2}\right) & 0\\ \Delta\,\mathrm{sinc}\left(\frac{\omega_t\Delta}{2}\right)\sin\left(\tilde{\theta}_t + \alpha_t + \frac{\omega_t\Delta}{2}\right) & 0\\ 0 & \Delta\end{bmatrix}\begin{bmatrix}v_t\\ \omega_t\end{bmatrix}\\ &\quad + \begin{bmatrix}\mathbf{r}_t - \mathbf{r}_{t+1}\\ \alpha_t - \alpha_{t+1}\end{bmatrix} + \mathbf{w}_t.\end{aligned} \tag{2}$$

Working in error space is advantageous because the optimal policy depends only on $\mathbf{e}_t$ rather than the full absolute state, and the reference trajectory period $T = 100$ makes the problem time-periodic, bounding the state space that must be considered.

The control objective is then to find a policy $\pi(t, \mathbf{e}_t)$ minimizing the expected discounted cumulative cost of tracking

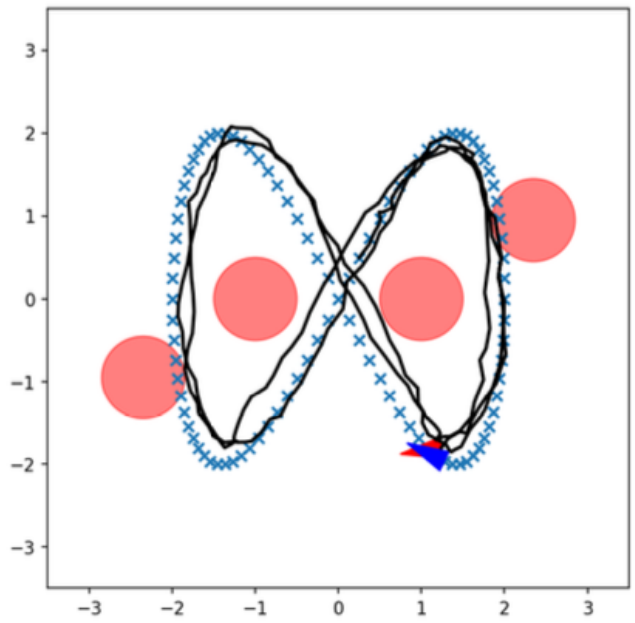


Fig. 1: The objective is to design a control policy for the differential drive robot (red triangle) to track the reference trajectory (blue crosses) while avoiding the two obstacles in the lemniscate path. Additionally, we would like to minimize the tracking error.

error and control effort:

$$V^*(\tau, \mathbf{e}) = \min_{\pi} \mathbb{E}\Bigg[\sum_{t=\tau}^{\infty} \gamma^{t-\tau}\Big(\tilde{\mathbf{p}}_t^\top \mathbf{Q}\tilde{\mathbf{p}}_t + q\big(1-\cos(\tilde{\theta}_t)\big)^2 + \mathbf{u}_t^\top \mathbf{R}\mathbf{u}_t\Big) \;\Big|\; \mathbf{e}_\tau = \mathbf{e}\Bigg]. \quad (3)$$

subject to the error dynamics (2), control constraints $\mathbf{u}_t = \pi(t, \mathbf{e_t}) \in \mathcal{U} := [0.1, 1] \times [-1, 1]$, and the obstacle avoidance constraint $\tilde{\mathbf{p}}_t + \mathbf{r}_t \in \mathcal{F}$. Let $\mathbb{S}^n_{++} = \{P \in \mathbb{R}^{n\times n} \mid P = P^\top,\ P \succ 0\}$ be the space of n-dimensional symmetric positive-definite matrices. Then, in Equation 3, $\mathbf{Q} \in \mathbb{S}^2_{++}$ defines the stage cost for deviating from the reference position trajectory $\mathbf{r_t}$, $q > 0$ is a scalar defining the stage cost for deviating from the reference orientation trajectory α_t, and $\mathbf{R} \in \mathbb{S}^2_{++}$ defines the cost for using excessive control effort. In the following section, we will describe two approaches for solving the objective in 3.

III. Technical Approach

Below, Sections III-A and III-B describe two methods for solving the objective in 3. Both approaches share the same cost structure and dynamics. The fundamental difference lies in how they handle the infinite-horizon stochastic problem. Certainty Equivalent Control avoids the stochasticity by acting like there is no noise and solving a deterministic finite-horizon optimization online at every timestep. This method is computationally expensive at runtime, but easy to implement using a premade nonlinear program solver. On the other hand, Generalized Policy Iteration solves the problem offline by discretizing the error state space into a grid and switching between a policy improvement step and a value update step. This results in cheap runtime operation since each control lookup is a table query, but requires significant upfront computation and careful choice of discretization.

A. Receding-Horizon Certainty Equivalent Control

CEC approximates the infinite-horizon stochastic problem (3) by setting the noise $\mathbf{w}_t = \mathbf{0}$ and solving a deterministic finite-horizon optimal control problem at each timestep τ:

$$\begin{aligned} V^*(\tau, \mathbf{e}) \approx \min_{\mathbf{u}_{\tau:\tau+T-1}} \quad & \mathrm{q}(\mathbf{e}_{\tau+T}) + \sum_{t=\tau}^{\tau+T-1} \gamma^{t-\tau}\ell(\mathbf{e}_t, \mathbf{u}_t) \\ \text{s.t.} \quad & \mathbf{e}_{t+1} = g(t, \mathbf{e}_t, \mathbf{u}_t, \mathbf{0}), \\ & t = \tau, \dots, \tau+T-1, \\ & \mathbf{u}_t \in \mathcal{U}, \\ & \tilde{\mathbf{p}}_t + \mathbf{r}_t \in \mathcal{F}. \end{aligned} \quad (4)$$

$$\ell(\mathbf{e}_t, \mathbf{u}_t) = \tilde{\mathbf{p}}_t^\top \mathbf{Q}\tilde{\mathbf{p}}_t + q(1-\cos(\tilde{\theta}_t))^2 + \mathbf{u}_t^\top \mathbf{R}\mathbf{u}_t. \quad (5)$$

where $\mathrm{q}(\mathbf{e})$ is a terminal cost penalizing the residual tracking error at the end.

The minimization variables are the control sequence $\mathbf{U} := [\mathbf{u}_\tau^\top, \dots, \mathbf{u}_{\tau+T-1}^\top]^\top \in \mathbb{R}^{2T}$, where each u_τ is the stacked linear and angular velocity at time step τ. Given the initial error state $\mathbf{e}_\tau$ and the noise-free dynamics model, the full error trajectory $\mathbf{E} := [\mathbf{e}_\tau^\top, \dots, \mathbf{e}_{\tau+T}^\top]^\top$ is a deterministic function of $\mathbf{U}$, so the problem reduces to an NLP over $\mathbf{U}$ of the form:

$$\min_{\mathbf{U}} \ c(\mathbf{U}, \mathbf{E}) \quad \text{s.t.} \quad \mathbf{U}_{lb} \le \mathbf{U} \le \mathbf{U}_{ub}, \quad \mathbf{h}_{lb} \le \mathbf{h}(\mathbf{U}, \mathbf{E}) \le \mathbf{h}_{ub}$$

The constraints enforce the actuation limits on the angular and linear velocity at every step. We enforce obstacle avoidance through a nonlinear inequality constraint requiring that the robot center $\tilde{\mathbf{p}}_t + \mathbf{r}_t$ stays outside each obstacle expanded by the robot radius:

$$\|\tilde{\mathbf{p}}_t + \mathbf{r}_t - \mathbf{c}_i\|^2 \ge (r_i + r_{\text{robot}} + r_{\text{margin}})^2, \quad \forall\, i,\ t = \tau, \dots, \tau+T$$

where $r_i = 0.5$, $r_{\text{robot}} = 0.3$, and r_{margin} is an additional buffer. These constraints are enforced at every step including the terminal state.

The NLP is solved using CasADi with the IPOPT interior-point solver. The problem is constructed once symbolically at initialization as a parametrized NLP with the current error state $\mathbf{e}_\tau$ and the reference trajectory segment $[\mathbf{r}_\tau, \dots, \mathbf{r}_{\tau+T}]$ passed as parameters. This is done by constructing the objective and constraint function as CasADi symbolic expressions through unrolling the noise-free error dynamics (2) forward through the horizon, accumulating the discounted stage costs and obstacle avoidance constraints at each step. Using this method allows the solver to obtain exact derivatives via automatic differentiation at each call and avoids the overhead of repeated symbolic compilation.

Another thing worth noting is that the sinc term in the error dynamics, which is undefined at $\omega = 0$, is handled via a CasADi if-else statement that switches to a second-order Taylor approximation, ensuring smooth gradients throughout. To further accelerate convergence, the solver is warm-started at each timestep by initializing $\mathbf{U}$ with the previous solution shifted forward by one step which provides a near-feasible starting point.

At runtime, once the optimal sequence $\mathbf{u}_\tau^*, \ldots, \mathbf{u}_{\tau+T-1}^*$ is obtained, only the first control $\mathbf{u}_\tau^*$ is applied to the system. At the next timestep the full optimization is repeated from the updated error state. This approach helps avoid unbounded error accumulation as the true noisy trajectory diverges from the predicted one.

B. Generalized Policy Iteration

GPI solves the same problem (3) by introducing an approximation elsewhere. Instead of working on the continuous space, it discretizes the error state space into a grid and iteratively computes an optimal policy. At runtime, a table lookup is used to give cheap control.

The error state space is discretized into a grid of $n_t \times n_x \times n_y \times n_\theta$ points. Since the reference trajectory is periodic with $T = 100$, only $n_t = 100$ distinct times need to be represented. THe position error is discretized uniformly over $[-3, 3]^2$ and the orientation error is discretized uniformly over $[-\pi, \pi]$, with wrap-around at the boundaries. The control space is also discretized into n_v linear velocity values over the allowable controls and n_w angular velocity values over the allowable controls. n_w is always chosen to be odd so that $\omega = 0$ is included. The value function is stored as a four-dimensional array of shape $(n_t, n_x, n_y, n_\theta)$ and indexed by nearest-neighbor lookup.

The algorithm does a pre-computation of transition probabilities and stage costs before any policy iteration happens. For each combination of time, state, and control, the noise-free next state is obtained using the error dynamics. The surrounding grid neighbors of this next state are obtained via trilinear interpolation, i.e. the fractional position of th next state within its grid cell is computed along each dimension, and then the 8 corner weights are set proportionally and normalized. The weights approximate $p_f(\mathbf{e}'|\mathbf{e}, \mathbf{u})$. The results are stored as integer index arrays and the float weight arrays of shape $(n_t, N, n_v, n_w, 8)$ where $N = n_x n_y n_\theta$, so that all of the following Bellman updates in the policy iteration step are just NumPy operations. The stage cost is similarly precomputed and stored as a (n_t, N, n_v, n_w) array.

Collision avoidance is ensured through adding a penalty $C_{\text{collision}}$ to the stage cost during the pre-computation stage. This penalty is added to any state where the robot body $\mathcal{B}_{0.3}(\tilde{\mathbf{p}}_t + \mathbf{r}_t)$ intersects an obstacle.

Once pre-computation is complete, GPI switches off between policy improvement and policy evaluation. In the policy improvement step, the following minimization is applied to all states simultaneously:

$$\pi(t, \mathbf{e}) = \underset{\mathbf{u} \in \mathcal{U}}{argmin} \left[\ell(\mathbf{e}, \mathbf{u}) + \gamma \sum_{\mathbf{e}'} P(\mathbf{e}'|\mathbf{e}, \mathbf{u})\, V(t+1, \mathbf{e}') \right]$$

The expected next value $\sum_{\mathbf{e}'} P(\mathbf{e}'|\mathbf{e}, \mathbf{u}) V(t+1, \mathbf{e}')$ is computed as a single operation over the pre-stored neighbor indices and weights, and the argmin over all $n_v \times n_w$ controls is taken for all N states simultaneously. In policy evaluation, the value function is then updated for `num_evals` sweeps forward over all T timesteps:

$$V(t, \mathbf{e}) \leftarrow \ell(\mathbf{e}, \pi(t, \mathbf{e})) + \gamma \sum_{\mathbf{e}'} P(\mathbf{e}'|\mathbf{e}, \pi(t, \mathbf{e}))\, V(t+1, \mathbf{e}')$$

The policy and value function are stored as arrays of shape (n_t, N) and $(n_t, n_x, n_y, n_\theta)$ respectively, and both iteration steps are fully vectorized over all states, making each outer iteration fast despite a large state space.

The most computationally expensive part of this pipeline is the pre-computation steps. To make this scale to larger grids, the work is parallelized across CPU cores using the python Ray package. In our implementation, each worker writes its results directly to a temporary `.npy` file on disk and returns only the file path. This avoids the potential memory overflow and disk spilling issues on larger grids. The parent process loads and concatenates all files after every worker finishes, and then deletes all the temporary `.npy` files. This keeps Ray's object story occupied only by a small set of strings regardless of grid size which allows scaling to finer resolutions without crashing.

C. Overall Technical Approach

In the spirit of properly comparing the two appraoches, we run multiple experiments for each algorithm that aim to characterize the effects of each algorithm's parameters on computation time, reference trajectory tracking, and collision avoidance.

For CEC, we evaluate the following:

1) **Effect of horizon length T.** We evaluate the CEC controller with varying horizon lengths T, holding all other parameters fixed. Since the NLP has $2T$ decision variables and $4(T+1)$ constraints, computation time grows with T. We expect longer horizons to improve collision avoidance and tracking near the lemniscate waist, where the robot must anticipate upcoming turns, at the cost of increased solve time per step.
2) **Effect of cost weights Q and q.** We vary the position cost and orientation weight while keeping all other variables fixed. A higher weight forces tighter tracking but reduces the robot's ability to navigate around the obstacles.
3) **Effect of collision margin** We vary the safety margin in III-A. A larger margin provides a more conservative buffer but also narrows the feasible passage between the center obstacles. This could potentially increase tracking error.

For GPI, we evaluate the following:

1) **Effect of grid resolution.** The accuracy of GPI depends highly on the grid resolution so we evaluate GPI on grids of size $13 \times 13 \times 13$, $17 \times 17 \times 17$, $21 \times 21 \times 21$, and $25 \times 25 \times 25$. In theory, a finer grid should allow better navigation through the narrow passages between obstacles and tighter turns of the lemniscate. However, the trade off is that pre-computation time and memory usage will increase quadratically.
2) **Effect of number of GPI iterations.** Keeping the grid fixed at $17 \times 17 \times 17$, we run GPI for $\{5, 10, 20, 50\}$ outer iterations and record error metrics after each. This will help characterize the convergence behavior and identifies the point of diminishing returns.
3) **Effect of number of policy evaluation sweeps.** With the number of outer iterations fixed at 20, we vary `num_evals` $\in \{1, 5, 10, 20, 50\}$. This controls how accurately the value function reflects the current policy before each improvement step. Too few sweeps can cause the policy to oscillate; too many waste computation that could be spent on additional improvement steps.

Finally, we will compare the error metrics of CEC and GPI using the best parameters from the previous experiments in order to elucidate which algorithm works best in this setting.

IV. Results

All the experiments were visualized using the numerical simulator. This simulator uses the discrete-time kinematic model with additive gaussian noise $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathrm{diag}(\boldsymbol{\sigma})^2)$ at each timestep. The following metrics were evaluated for each experiment:

1) **Cumulative Translational Error** is defined as the sum of Euclidean distances between the robot's position and the reference position at each timestep over the full simulation of $N = 240$ steps.
2) **Cumulative rotational error** is defined as the sum of absolute heading deviations in radians between the robot's orientation and the reference orientation at each timestep, with angles wrapped to $[-\pi, \pi]$.
3) **Average iteration time** (ms) measures the average time per control step during the simulation loop. This is meant to measure the online computational cost of each controller. For CEC this involves the full NLP solve, while for GPI it reflects only the table lookup.
4) **Collision count** reports the number of timesteps at which the robot center $\mathbf{p}_t$ comes within $r_i + r_{\text{robot}} = 0.8$m of any obstacle center, indicating a physical intersection between the robot body and an obstacle. This is evaluated via visualization.
5) **Precompute Time (s)** only applies to GPI, and reports the total time required for pre-computation of transition matrices and policy iterations. Note that this is a one time cost and does not affect runtime performance.

A. Certainty Equivalent Control

*1) **Experiment 1 - Effect of Horizon Length**:* Using the parameters in Table I, the results in Table II were achieved. See Appendix A for the resulting trajectories.

TABLE I: Parameters used for the CEC controller in Experiment 1.

Category	Parameter	Value	Description
Cost	Q	$\mathrm{diag}(10, 10)$	Position error weight
	q	30	Orientation error weight
	R	$\mathrm{diag}(0.1, 0.1)$	Control effort weight
Prediction	T	–	Experiment variable
	γ	0.995	Discount factor
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	10^{-3}	Collision margin

TABLE II: Effect of prediction horizon on controller performance.

T	Trans. Error	Rot. Error	Avg. Time (ms)	Collisions
1	152.490	85.981	1.81	0
5	53.713	53.082	2.75	0
10	54.286	59.458	5.38	0
15	53.767	54.996	10.16	0
20	53.567	51.945	19.88	0

The theory says that increasing the prediction horizon T should improve controller performance because a longer horizon allows the optimizer to anticipate upcoming turns and obstacles with more accruacy. This would allow the planner to create smoother avoidance maneuvers rather than reacting at the last moment. Since the NLP optimizes a finite-horizon approximation of the infinite-horizon problem (3), increasing T should also provide a closer approximation to the true optimal solution. Specifically, for the lemniscate trajectory, the regions with sharp turns are where increasing the horizon should provide the greatest improvement. The primary cost of increasing T is computational and the cost scales approximately linearly with T.

The results in Table II largely confirm the computational prediction but challenge the accuracy prediction. We found that computation time does increase with T from 1.81ms at $T = 1$ to 19.88ms at $T = 20$. However, the error does not keep decreasing as T increases. We see a notable drop from $T = 1$ to $T = 5$, then the following longer horizons achieve nearly identical error metrics. This indicates that a single-step greedy controller is very suboptimal, but that extending the horizon beyond $T = 5$ yields diminishing returns. This is likely because the lemniscate is smooth and period, and the obstacles are sufficiently separated from the reference path, meaning that even a shorter horizon controller can perform well. Additionally, all controllers get zero collisions because this constraint is hard coded into the NLP. Another thing worth noting is that the rotation error is non-monotone. This is likely

TABLE IV: Effect of cost weights Q and q on controller performance.

$\mathbf{Q}$	q	$\mathbf{R}$	Trans. Error	Rot. Error	Avg. Time (ms)	Collisions
$\mathrm{diag}(0.01, 0.01)$	0.01	$\mathrm{diag}(0.1, 0.1)$	444.235	190.348	2.94	0
$\mathrm{diag}(1, 1)$	1	$\mathrm{diag}(0.1, 0.1)$	52.731	48.485	2.29	0
$\mathrm{diag}(5, 5)$	5	$\mathrm{diag}(0.1, 0.1)$	56.789	67.213	2.90	0
$\mathrm{diag}(10, 10)$	10	$\mathrm{diag}(0.1, 0.1)$	53.295	61.995	2.56	0
$\mathrm{diag}(20, 20)$	20	$\mathrm{diag}(0.1, 0.1)$	54.607	63.611	2.54	0
$\mathrm{diag}(50, 50)$	50	$\mathrm{diag}(0.1, 0.1)$	52.456	59.632	2.55	0
$\mathrm{diag}(100, 100)$	100	$\mathrm{diag}(0.1, 0.1)$	54.352	60.692	3.00	0

because of stochastic noise in the simulator rather than any meaningful structural effect. Based on these results $T - 5$ is the clear practical choice because it captures nearly all the benefits of higher horizons with minimal computation time compared to $T = 20$.

*2) **Experiment 2 - Effect of Cost Weights**:* Using the parameters in Table III, the results in Table IV were achieved. See Appendix B for the resulting trajectories.

TABLE III: Parameters used for the CEC controller in Experiment 2.

Category	Parameter	Value	Description
Cost	Q	–	Experiment Variable
	q	–	Experiment Variable
	R	$\mathrm{diag}(0.1, 0.1)$	Control effort weight
Prediction	T	5	Prediction Horizon
	γ	0.995	Discount factor
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	10^{-3}	Collision margin

The results in Table IV reveal a clear threshold effect in the cost weights. At ($\mathbf{Q} = \mathrm{diag}(0.01, 0.01)$, $q = 0.01$), the controller ignores tracking error and minimizes control effort, resulting very high translation rotation errors. Once the tracking weights are raised to $\mathbf{Q} = \mathrm{diag}(1, 1)$, $q = 1$, the tracking improves dramatically and we see little improvement thereafter. Across all other increased weights, the errors remain essentially flat and any slight improvements can be attributed to the simulator's noise rather than a structural benefit. Another thing worth noting is that collisions are zero for all weights, including the extreme $\mathbf{Q} = \mathrm{diag}(100, 100)$ case, suggesting that for this environment, the collision avoidance constraints in the NLP are sufficient for preventing collisions. These results suggest that CEC is robust to the choice of tracking weights over several orders of magnitude, provided they are above a minimum threshold, and that the hard obstacle constraints rather than the soft cost terms are the primary mechanism for collision avoidance in this formulation.

*3) **Experiment 3 - Effect of Collision Margin**:* Using the parameters in Table V, the results in Table VI were achieved. See Appendix C for the resulting trajectories.

TABLE V: Parameters used for the CEC controller in Experiment 3.

Category	Parameter	Value	Description
Cost	Q	$\mathrm{diag}(10, 10)$	Position Error Weight
	q	10	Orientation Error Weight
	R	$\mathrm{diag}(0.1, 0.1)$	Control effort weight
Prediction	T	5	Prediction Horizon
	γ	0.995	Discount factor
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	–	Experiment Variable

TABLE VI: Effect of collision margin on controller performance.

r_{margin}	Trans. Error	Rot. Error	Avg. Time (ms)	Collisions
0	54.906	64.819	2.56	0
$1 \times 10^{-}4$	51.6002	57.265	2.56	0
$1 \times 10^{-}3$	55.159	59.822	2.62	0
$1 \times 10^{-}2$	53.998	63.919	2.56	0
$1 \times 10^{-}1$	177.540	139.134	4.45	0
1	441.534	179.781	5.21	0

From the results, we can see that the collision margin has a negligible effect on performance until we reach large values relative to the available free space. For margins up to $r_{\text{margin}} = 10^{-2}$, both errors remain essentially flat, with any deviations attributable to simulator noise, suggesting that small safety buffers don't really do much in the NLP case.

However, at $r_{\text{margin}} = 0.1$, the tracking error increases sharply. At $r_{\text{margin}} = 1.0$, each obstacle has a radius of essentially 1.8m, making the controller produce errors comparable to the near-uncontrolled case. This is because the inflated obstacles block most of the lemniscate path, forcing the robot far from the reference in order to enforce obstacle avoidance. At each level of safety margin, zero collisions were observed, further enforcing that the NLP constraints alone are sufficient to guarantee obstacle avoidance. These results suggest that any $r_{\text{margin}} \leq 10^{-2}$ is practical.

Across all experiments, CEC demonstrates consistent obstacle avoidance due to the hard constraint formulation in the NLP, which enforces obstacle avoidance as a requirement rather than a soft penalty. It was observed that tracking performance is primarily sensitive to the prediction horizon

TABLE VIII: Effect of grid resolution on GPI controller performance.

$n_x \times n_y \times n_\theta$	Trans. Error	Rot. Error	Collisions	Precompute (s)	Avg. Time (ms)
$13 \times 13 \times 13$	178.891	97.903	0	26.6	0.12
$17 \times 17 \times 17$	158.519	103.041	1	56.07	0.11
$21 \times 21 \times 21$	112.985	97.491	0	218.38	0.38
$25 \times 25 \times 25$	108.694	94.385	0	1084.94	0.48

and tracking cost weights. In all experiments, a thresholding effect was observed, where once the test variable was taken beyond a certain value, performance either degraded (safety margin experiment) or improved (prediction horizon and cost experiments) drastically, then plateaued. Overall, CEC has proven to be a reliable and computationally efficient controller for this problem with $T = 5$, $\mathbf{Q} = \text{diag}(1, 1)$, $q = 1$, and $r_{\text{margin}} = 10^{-3}$ representing a well-performing and practically deployable parameter configuration.

B. Generalized Policy Iteration

1) ***Experiment 1 - Effect of Grid Resolution****:* Using the parameters in Table VII, the results in Table VIII were achieved. See Appendix D for the resulting trajectories.

TABLE VII: Parameters used for the GPI controller in Experiment 1.

Category	Parameter	Value	Description
Cost	$\mathbf{Q}$	$\text{diag}(10, 10)$	Position weight
	q	10	Orientation weight
	$\mathbf{R}$	$\text{diag}(0.01, 0.01)$	Control weight
Discretization	n_x, n_y	–	Experiment Variables
	n_θ	–	Experiment Variables
	n_v	5	Linear velocity grid points
	n_ω	9	Angular velocity grid points
Policy	γ	0.995	Discount factor
	N_{iters}	30	GPI iterations
	N_{evals}	20	Evaluation sweeps
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	$1e-4$	Collision margin (m)
	C_{col}	3000	Collision penalty

Table VIII reveals a clear improvement in tracking performance as grid resolution increases mainly in the translational component. Translation error drops from 178.9 at $13 \times 13 \times 13$ to 108.7 at $25 \times 25 \times 25$ — a 39%, however, the rotational component remains essentially flat, with any improvements attributable to simulator noise. This confirms the theoretical prediction that finer resolutions increase narrow passage navigation performance. The single collision recorded in the $17 \times 17 \times 17$ case is likely due to noise rather than a structural property of that resolution, as both courser and finer grids had zero collisions.

The most drastic tradeoff in grid resolution is seen in the computational costs. The precompute time increases drastically as the grid resolution becomes finer and finer. Precompute time seems to grow super-linearly because the transition matrix has shape $(n_t, N, n_v, n_w, 8)$, so memory and compute both grow as $O(n^3)$ in the grid size. Runtime per step remains nearly constant across all resolutions, since control selection is always a simple table lookup regardless of grid size.

The results suggest that $21 \times 21 \times 21$ offers the best practical tradeoff: it achieves 112.9 translation error and zero collisions at a precompute time of 218s. This is virtually the same error as the $25 \times 25 \times 25$ grid at almost one fifth the precompute cost. The error improvements between $21 \times 21 \times 21$ and $25 \times 25 \times 25$ are modest ($112.9 \rightarrow 108.7$), suggesting that further resolution increases would yield diminishing returns — the remaining error is likely dominated by the irreducible effect of motion noise rather than grid coarseness.

2) ***Experiment 2 - Effect of Number of GPI iterations****:* Using the parameters in Table IX, the results in Table X were achieved. See Appendix E for the resulting trajectories.

TABLE IX: Parameters used for the GPI controller in Experiment 2.

Category	Parameter	Value	Description
Cost	$\mathbf{Q}$	$\text{diag}(10, 10)$	Position weight
	q	10	Orientation weight
	$\mathbf{R}$	$\text{diag}(0.01, 0.01)$	Control weight
Discretization	n_x, n_y	17	Position grid points
	n_θ	17	Orientation grid points
	n_v	5	Linear velocity grid points
	n_ω	9	Angular velocity grid points
Policy	γ	0.995	Discount factor
	N_{iters}	–	Experiment Variable
	N_{evals}	20	Evaluation sweeps
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	$1e-4$	Collision margin (m)
	C_{col}	3000	Collision penalty

TABLE X: Effect of GPI iterations on GPI controller performance.

N_{iters}	Trans. Error	Rot. Error	Collisions	Precompute (s)	Avg. Time (ms)
5	185.712	115.322	1	16.03	0.12
10	150.861	98.910	3	23.98	0.12
20	176.763	118.831	1	40.08	0.14
50	144.440	91.706	1	87.50	0.14

TABLE XII: Effect of Number of Policy Sweeps on GPI controller performance.

N_{evals}	Trans. Error	Rot. Error	Collisions	Precompute (s)	Avg. Time (ms)
1	153.869	90.431	1	26.64	0.26
5	146.723	97.331	2	30.86	0.10
10	142.617	94.633	0	39.90	0.15
20	168.7522	119.229	2	55.18	0.15
50	145.726	93.934	2	149.72	0.14

The results in Table X show a weak and non-monotone relationship between the number of GPI outer iterations and tracking performance. This behavior is a little unexpected from a theoretical standpoint, since GPI is guaranteed to produce a policy no worse than the previous iteration at each step. One could attribute the deviations to stochastic noise in the simulator because with motion noise $\boldsymbol{\sigma} = [0.04, 0.04, 0.004]^\top$, the cumulative error over only 240 timesteps has significant variance. However, these deviations seem to be a little large compared to what we have seen in past experiments.

The pre-compute time grows almost linearly with the number of iterations which is consistent with the fixed per-iteration cost of the policy improvement and evaluation sweeps. The collision counts are also mostly consistent between settings, with at most 3 collisions recorded at $N_{\text{iters}} = 10$. However, note that these are again likely stochastic rather than structural. Overall, the results suggest that on a $13\times13\times13$ grid the policy converges quickly and additional iterations provide limited benefit — the dominant source of error at this resolution is grid coarseness rather than insufficient policy optimization, consistent with the findings of Experiment 1.

*3) **Experiment 3 - Effect of Number of policy evaluation sweeps**:* Using the parameters in Table XI, the results in Table XII were achieved. See Appendix F for the resulting trajectories.

TABLE XI: Parameters used for the GPI controller in Experiment 3.

Category	Parameter	Value	Description
Cost	$\mathbf{Q}$	$\text{diag}(10, 10)$	Position weight
	q	10	Orientation weight
	$\mathbf{R}$	$\text{diag}(0.01, 0.01)$	Control weight
Discretization	n_x, n_y	17	Position grid points
	n_θ	17	Orientation grid points
	n_v	5	Linear velocity grid points
	n_ω	9	Angular velocity grid points
Policy	γ	0.995	Discount factor
	N_{iters}	30	GPI iterations
	N_{evals}	–	Experiment Variable
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	$1e-4$	Collision margin (m)
	C_{col}	3000	Collision penalty

The results in Table XII show that the number of policy evaluation sweeps N_{evals} has minimal impact on tracking performance, with all configurations producing translation errors in the range 142–169 and rotation errors in the range 90–119. As in the previous experiment, the relationship is non-monotone meaning that performance does not improve consistently as N_{evals} increases. Again, the variance across runs is likely because of stochastic noise in the simulator that results in high variance across 240 time steps. The single exception is $N_{\text{evals}} = 10$, which we observed to have zero collisions and the lowest translational error, though this is, again, unlikely a structural advantage of that value.

Precompute time scales approximately linearly with N_{evals}, since each additional sweep requires a full forward pass over all $T = 100$ timesteps and N states. Runtime per step remains unaffected at around 0.1–0.3ms. These results suggest that on this grid resolution, a single policy evaluation sweep is nearly as effect as 50, implying that the value function converges quickly relative to the policy improvement step at this resolution. Using the results from this experiment and experiment 2, we can conlude that on course grids, the GPI algorithm converges in few iterations with few evaluation sweeps and the remaining error is attributable to discretization error rather than insufficient optimization. Therefore, using more iteration sweeps is only advantageous when coupled with a finer resolution.

C. Overall Comparison

Having characterized the effect of key parameters for each algorithm independently, we now compare the two controllers directly under their best-performing parameter configurations. For each controller, we use the best parameters from the experiments which are shared again in Tables XIII and XIV for convenience. Since the simulator applies independent Gaussian motion noise at every timestep, cumulative error metrics vary across runs. To obtain statistically reliable estimates, each controller is evaluated over 5 independent trials and we report the mean and standard deviation of translation error, rotation error, and collision count. The mean translational and rotational error along with the mean collision count are shared in Table XV. The images in Figure 2 are representative trajectories for each algorithm.

TABLE XIII: Parameters used for the CEC controller in Overall Comparison.

Category	Parameter	Value	Description
Cost	Q	diag(1,1)	Position cost weight
	q	1	Orientation cost weight
	R	$\text{diag}(0.1, 0.1)$	Control effort weight
Prediction	T	5	Prediction Horizon
	γ	0.995	Discount factor
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	10^{-4}	Collision margin

TABLE XIV: Parameters used for the GPI controller in Overall Comparison.

Category	Parameter	Value	Description
Cost	$\mathbf{Q}$	$\text{diag}(10, 10)$	Position weight
	q	10	Orientation weight
	$\mathbf{R}$	$\text{diag}(0.01, 0.01)$	Control weight
Discretization	n_x, n_y	25	Position grid points
	n_θ	25	Orientation grid points
	n_v	5	Linear velocity grid points
	n_ω	9	Angular velocity grid points
Policy	γ	0.995	Discount factor
	N_{iters}	50	GPI iterations
	N_{evals}	20	Policy Sweeps
Safety	r_{robot}	0.3	Robot radius (m)
	r_{margin}	$1e-4$	Collision margin (m)
	C_{col}	3000	Collision penalty

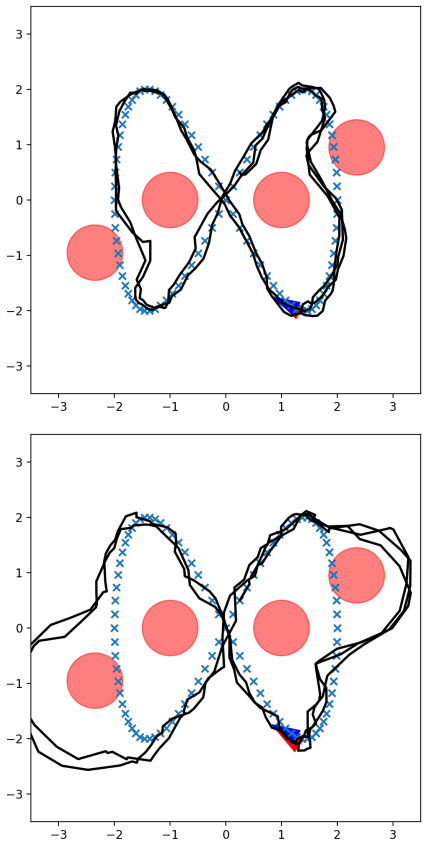


Fig. 2: CEC (top) best parameters trajectory versus GPI (bottom) best parameters trajectory.

TABLE XV: Head-to-head comparison of CEC and GPI controllers averaged over 5 independent trials.

Controller	Avg. Trans. Error	Avg. Rot. Error	Avg. Collisions
CEC	53.48 ± 1.87	50.87 ± 4.1	0 ± 0
GPI	107.554 ± 4.135	01.441 ± 4.125	0 ± 0

Looking at the results in Table XV, we confirm our intuition that CEC outperforms GPI in tracking accuracy, achieving roughly half the average translation and rotation error. Both controllers achieve zero collisions on average, demonstrating that either approach can successfully navigate this environment safely despite their different mechanisms for obstacle avoidance. The representative trajectories in Figure 2 illustrate the difference in performance qualitatively. CEC tracks the lemniscate tightly across both lobes, while GPI produces a looser trajectory that follows the general shape but noticeably deviates in the high-curvature regions.

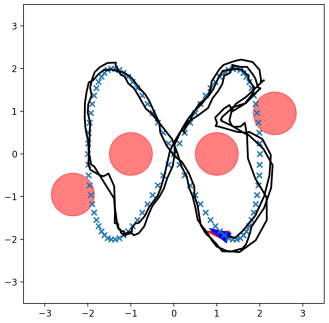
The performance gap is primarily attributable to the courseness of the GPI grid, which is a fundamental limitation of the algorithm. CEC operators in the continuous control space and reoptimizes at every step with full knowledge of the current error state, while GPI is constrained to discrete actions available on its grid and cannot adapt online to specific noise realizations. It is worth noting that GPI's main advantage is its average computation time which is 0.14ms per step versus 2.20ms for CEC, making it far better for high-frequency control situations such as legged robots. With a sufficiently fine grid and enough iterations, GPI would be expected to perform much better as its policy more accurately approximates the true optimal solution.

V. Conclusion

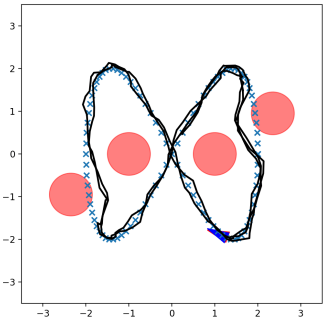
This project implemented and compared two approaches for safe trajectory tracking on a differential-drive robot: receding-horizon Certainty Equivalent Control (CEC) and Generalized Policy Iteration (GPI). Both controllers were evaluated on the task of tracking a lemniscate reference trajectory while avoiding four circular obstacles under Gaussian motion noise, and characterized through a series of controlled experiments varying key algorithmic parameters. CEC proved to a robust and accurate controller with its performance largely reliant on parameters that displayed a thresholding effect. However, it could not overcome the irreducible tracking error introduced by motion noise, which it does not account for by design. GPI offered a superior computation profile at runtime, but achieved almost double the tracking error. Its performance was largely dependent on the grid resolution.

Ultimately, the results suggest that CEC is the preferred choice when tracking accuracy and safety are paramount and real-time computation is available, while GPI becomes attractive in resource-constrained settings where online computation must be minimized.

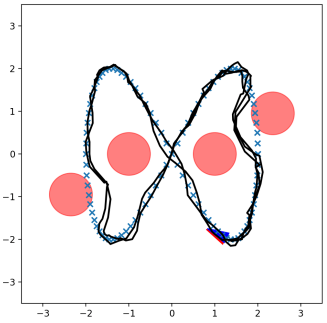
APPENDIX A
CEC EXPERIMENT 1



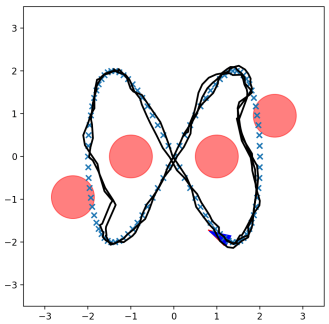
(a) Experiment 1, $T = 1$



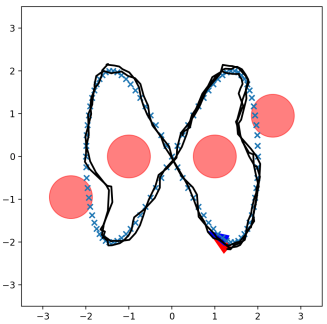
(b) Experiment 1, $T = 5$



(c) Experiment 1, $T = 10$



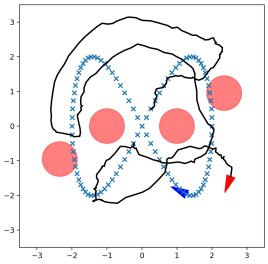
(d) Experiment 1, $T = 15$



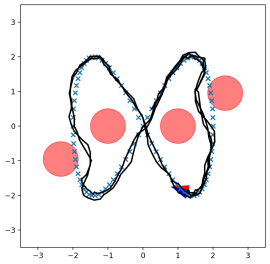
(e) Experiment 1, $T = 20$

Fig. 1: CEC Experiment 1: Varied prediction horizons. The images display the overall trajectories (black) of the robot (red triangle) tracking a reference lemniscate (blue crosses/blue triangle) under different prediction horizons.

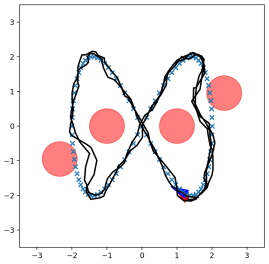
APPENDIX B
CEC EXPERIMENT 2



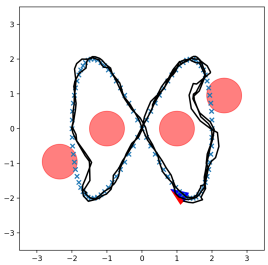
(a) $Q = \text{diag}(0.01, 0.01)$, $q = 0.01$



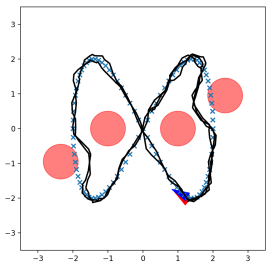
(b) $Q = \text{diag}(1, 1)$, $q = 1$



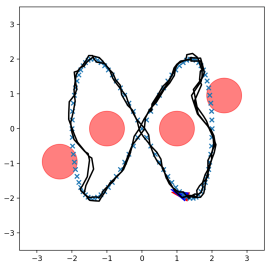
(c) $Q = \text{diag}(5, 5)$, $q = 5$



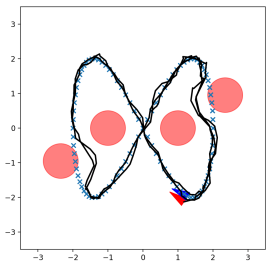
(d) $Q = \text{diag}(10, 10)$, $q = 10$



(e) $Q = \text{diag}(20, 20)$, $q = 20$



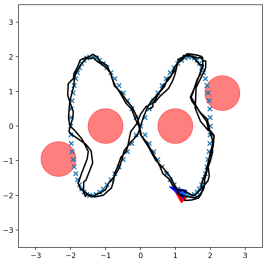
(f) $Q = \text{diag}(50, 50)$, $q = 50$



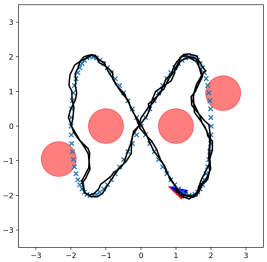
(g) $Q = \text{diag}(100, 100)$, $q = 100$

Fig. 2: CEC Experiment 2: Effect of Cost Weights. The images display the overall trajectories (black) of the robot (red triangle) tracking a reference lemniscate (blue crosses/blue triangle) under different costs.

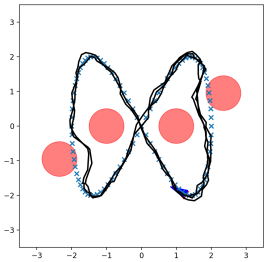
APPENDIX C
CEC EXPERIMENT 3



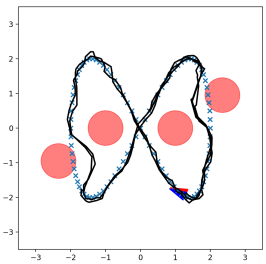
(a) $r_{margin} = 0$



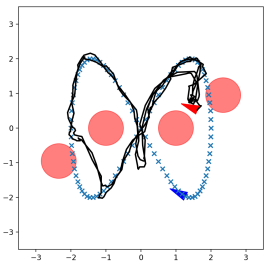
(b) $r_{margin} = 1 \times 10^{-4}$



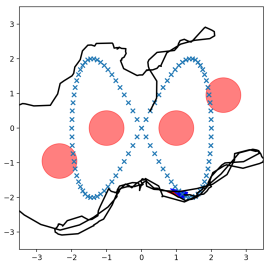
(c) $r_{margin} = 1 \times 10^{-3}$



(d) $r_{margin} = 1 \times 10^{-2}$



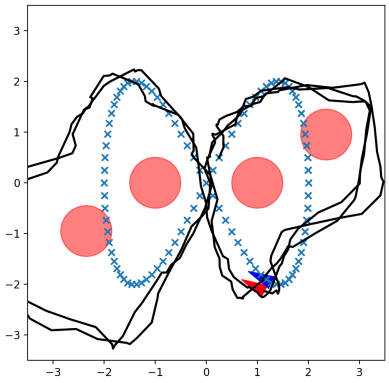
(e) $r_{margin} = 1 \times 10^{-1}$



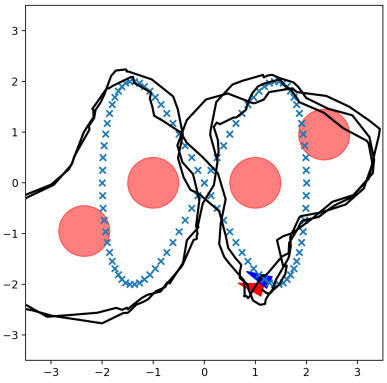
(f) $r_{margin} = 1$

Fig. 3: CEC Experiment 3: Effects of Collision Margin. The images display the overall trajectories (black) of the robot (red triangle) tracking a reference lemniscate (blue crosses/blue triangle).

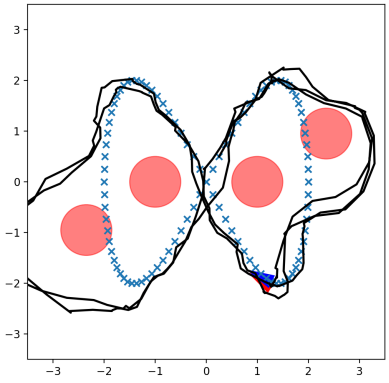
APPENDIX D
GPI EXPERIMENT 1



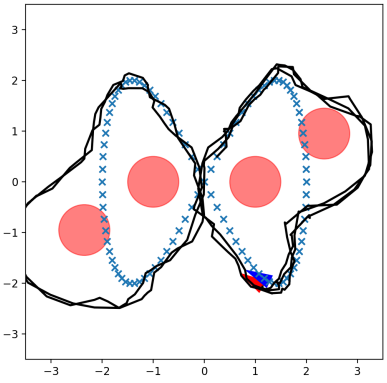
(a) 13x13x13



(b) 17x17x17



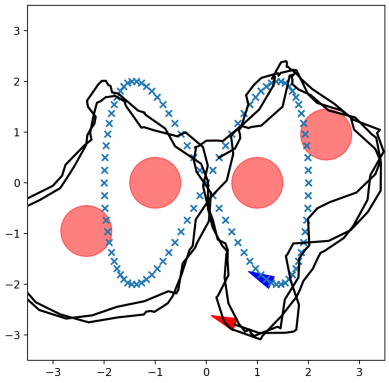
(c) 21x21x21



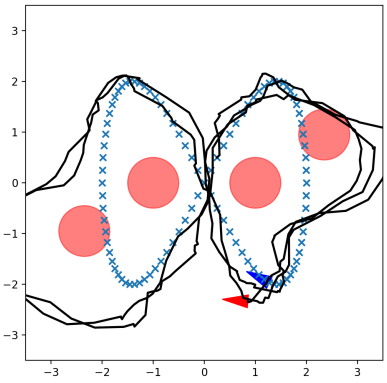
(d) 25x25x25

Fig. 4: GPI Experiment 1: Effects of grid resolution. The images display the overall trajectories of the robot tracking the reference trajectory under different grid discretizations.

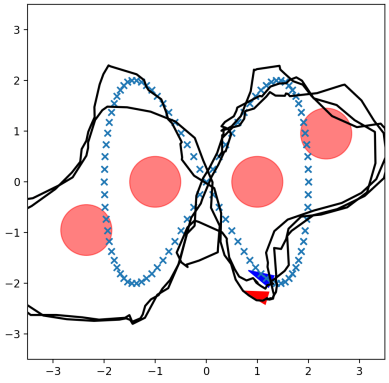
APPENDIX E
GPI EXPERIMENT 2



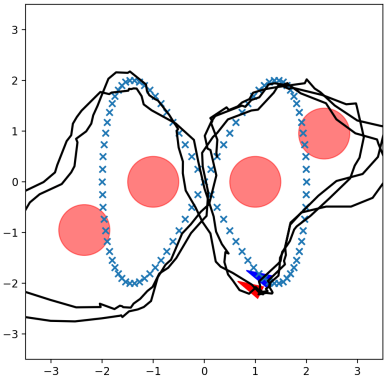
(a) $N_{iters} = 5$



(b) $N_{iters} = 10$



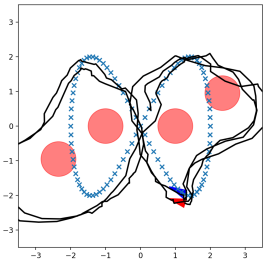
(c)$N_{iters} = 20$



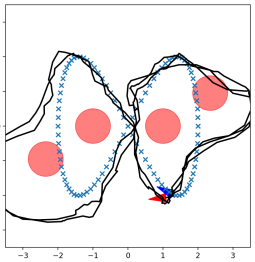
(d) $N_{iters} = 50$

Fig. 5: GPI Experiment 2: Effect of Number of GPI iterations. The images display the overall trajectories of the robot tracking the reference trajectory under different iteration counts.

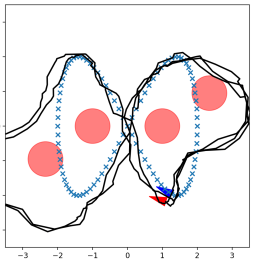
APPENDIX F
GPI EXPERIMENT 3



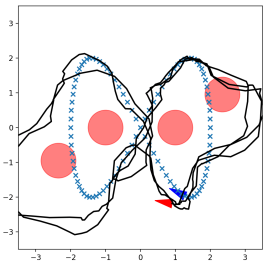
(a) $N_{evals} = 1$



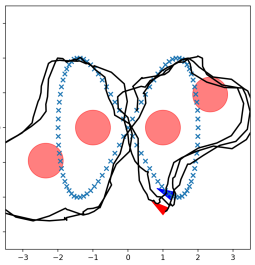
(b) $N_{evals} = 5$



(c) $N_{evals} = 10$



(d) $N_{evals} = 20$



(e) $N_{evals} = 50$

Fig. 6: GPI Experiment 3: Effects of number of policy sweeps in each iteration. The images display the overall trajectories of the robot tracking the reference trajectory under different policy sweep counts.